



CareerHubX

Assignment 3.2 — Walkthrough

Multi-step application wizard, document uploads, branding & the testing suite

Next.js 15 · React 19 · TypeScript · Tailwind v4 · TanStack Query · nuqs · Auth.js

Testing: Vitest · Testing Library · user-event · MSW · jsdom · GitHub Actions

Prepared 2026-06-30 · Branch `CareerHub-3.1`

0 · Overview & what shipped

This document walks through everything delivered in this session, how it was built, the key code, how the UI behaves, and a one-page terminal runbook (frontend + backend + Docker + tests).

Two bodies of work

Assignment 3.2 — Feature build	Assignment 3.2 — Testing
① 5-step Job Application Wizard ② Document uploads (PDF, conditional, consent) ③ CareerHubX branding ④ Backend contract / gap list	Part 1 written decisions · Part 2 Vitest setup · Part 3 wizard tests · Part 4 MSW network tests · Part 5 GitHub Actions CI

Verification status

Check	Command	Result
Type check	<code>npx tsc --noEmit</code>	0 errors
Production build	<code>npm run build</code>	compiles (all routes)
Tests	<code>npm run test:run</code>	12 passed / 12
CI	GitHub Actions on push/PR → main	workflow committed

Backend & Docker note. The live ASP.NET/PostgreSQL backend (`careerhub-api`) is a separate repository and is **not in this workspace**, so it could not be built or run via Docker from here. The testing assignment does not need it (tests use MSW + jsdom). Where the new features need persistence the frontend is wired to the *real contract names* and falls back to `localStorage` (clearly flagged) so the flow is fully functional for a demo. The exact endpoints/DTOs the backend must add are specified in `docs/BACKEND-GAPS.md` . The terminal runbook (§6) includes the Docker commands for when the backend repo is present.

1 · The 5-step application wizard

Feature 1 `src/components/apply/JobApplicationWizard.tsx` · route `/apply/[jobId]`

What it does



#	Step	Collects
1	Personal information	full name(s), surname, gender, race (optional + “Prefer not to say”), date of birth, disability (+details if Yes), nationality
2	Contact details	email, phone, alternate phone, address, city, province (SA + Other), postal code
3	Qualifications	highest qualification, institution, year, employment status, years of experience, LinkedIn/portfolio
4	Job-specific	motivation (≥100 chars), expected salary (ZAR), notice period, start date, willing to relocate
5	Documents & consent	required PDFs (\$2) + three consent checkboxes

How it’s built

- **One Zod schema** (`src/lib/apply-wizard.ts`) with per-step field lists; **React Hook Form** validates only the current step before Next.
- **nuqs** keeps position in the URL (`?step=`) so a refresh resumes there.
- **Save as Draft** on every step (localStorage fallback) + profile pre-fill.
- **Accessible progress bar** (`WizardProgress.tsx`): `aria-current="step"` , visited steps are keyboard-focusable buttons, collapses to “Step N of M” under `md` .
- **Submit** → AlertDialog confirm → `sonner` toast → redirect to a printable confirmation page.

Key code — per-step validation gate

```
// trigger() validates ONLY the current step's fields before advancing
async function next() {
  const valid = await trigger(STEP_FIELDS[current], { shouldFocus: true });
  if (!valid) return;           // blocked – errors render inline
  persistDraft(current + 1);    // Save-as-Draft semantics on every advance
  goto(current + 1);
}
```

Key code — URL as a durable mirror (robust + testable)

```
const [urlStep, setUrlStep] = useQueryState("step",
  parseAsInteger.withDefault(0).withOptions({ history: "replace" }));
const [current, setCurrent] = useState(() => clampStep(urlStep ?? 0)); // read once
function goto(target) { setCurrent(clampStep(target)); void setUrlStep(target); } // write on change
```

2 · Documents step (PDF uploads + consent)

Feature 2

src/components/apply/DocumentUpload.tsx

How the UI behaves

 **CV / Résumé** alice@example.com → **aliceexamplecom-Cv.pdf** · 248 KB

[View](#)  Remove

 **Driver's Licence** Drag & drop or browse · PDF only, up to 3 MB

[Upload](#)

"No special characters or spaces in filenames — uploads are renamed automatically to `{userId}-{document}.pdf`."

- Drag-and-drop or click; **PDF only, max 3 MB**, validated with **inline** errors (not toasts).
- Display name is server-style sanitised; size shown in KB; **Remove** is guarded by an AlertDialog.

Conditional document matrix

Document	Required	Condition
ID Document (SA) or Passport	Yes	by nationality (step 1)
Matric Results	Yes	always
Tertiary Qualification	Conditional	if step-3 qualification is tertiary
Driver's Licence	Conditional	if the job requires it
Motivation Letter / CV	Yes	always

Key code — required-set is derived from answers

```
const reqDocs = requiredDocuments({
  isSouthAfrican: nationality === "South African",
  hasTertiary: isTertiary(highestQualification),
  jobRequiresLicence: job.requiresDriversLicence,
});
const allDocsUploaded = reqDocs.every((d) => docs[d.type]);
// Submit is disabled until allDocsUploaded && all three consents are ticked.
```

Consent (all required before submit): *information is accurate* · *consent to data processing per the Privacy Policy* · *agree to be contacted by employers*. A </privacy> page was added for the link.

3 • CareerHubX branding

Feature 3

`src/components/brand/Logo.tsx` • `src/app/icon.svg`

An inline-SVG brand mark (a person silhouette inside a magnifying-glass ring) and a two-tone “CareerHubX” wordmark, scaling and theming via brand tokens — no PNG.



- `<Logo>` (mark + wordmark, `onDark` variant) and `<LogoMark>` (icon only).
- Wired into the navbar, footer, auth shell, and the `app/icon.svg` favicon.
- Page titles use a `%s • CareerHubX` template + OpenGraph defaults in `layout.tsx`.

4 · Testing — setup & the adaptation

The key insight. The assignment targets the *old* 3-step wizard (`useSession` , HTTP submit, a “review” step). This session had replaced it with the 5-step wizard (auth as a `user` prop, `localStorage` submit, separate confirmation page). So the demo pattern was **adapted to the real components** — which the brief explicitly rewards.

Assignment expectation	Real component tested
Wizard navigation / validation	<code>JobApplicationWizard</code>
Auth gate	<code>/apply/[jobId]</code> page (gating moved to the page)
MSW submit (happy / error)	<code>ApplicationForm</code> (the component that actually POSTs + resets)
AlertDialog confirm	<code>CloseJobButton</code> (Server Action mocked — MSW can’t intercept actions)
Draft persistence	<code>JobApplicationWizard</code> via real jsdom localStorage

Part 2 — the provider harness

```
// src/test/utils.tsx
vi.mock("next-auth/react", () => ({ useSession: vi.fn(), /* ... */ }));
export function renderWithProviders(ui, { session = candidateSession, searchParams = "" } = {}) {
  vi.mocked(useSession).mockReturnValue({ data: session, status: session ? "authenticated" : "unauthenticated" });
  return render(ui, { wrapper: ({ children }) =>
    <NuqsTestingAdapter searchParams={searchParams}>
      <QueryClientProvider client={makeClient()}>/* retry:false */
        <AuthProvider>{children}</AuthProvider>
      </QueryClientProvider>
    </NuqsTestingAdapter> });
}
```

Part 4 — MSW: real HTTP interception

```
// happy path returns 201; error path overrides per-test:
server.use(http.post(`${API}/api/applications`, () => new HttpResponse(null, { status: 500 })));
// → assert a server-error alert appears AND the typed value is still present (no reset)
expect(await screen.findByRole("alert")).toBeInTheDocument();
expect(screen.getByDisplayValue("Alice Candidate")).toBeInTheDocument();
```

5 · Testing — results & decisions

Suite (12 tests, 5 files) — all green

File	Tests
<code>ApplicationWizard.test.tsx</code>	smoke · blocks-when-invalid · advances to step 2 · Back preserves values
<code>apply-gate.test.tsx</code>	sign-in prompt when unauthenticated · wizard renders for a candidate
<code>ApplicationForm.test.tsx</code>	MSW happy path (confirmation) · MSW 500 (values kept)
<code>CloseJobButton.test.tsx</code>	AlertDialog opens · confirm runs the action + shows closed state
<code>draft-persistence.test.tsx</code>	restores a saved draft + banner · writes a draft on "Save as Draft"

Written decisions (condensed — full text in README)

Q1 high-value: step gating, Back-preserves-values, document/consent gate, submit success/error. **NOT worth testing:** Tailwind class strings & DOM shape — they break on restyles and assert nothing the user feels.

Q1C / localStorage: use **real jsdom** localStorage — the behaviour *is* the write→reload→restore round-trip; a spy only proves a method was called, not that the value comes back into the form.

Q2 session: `vi.mock("next-auth/react")` (mocks just the hook, keeps the tree real). Caveat: our components read `AuthContext`, so the gate test plants `careerhub.session` in localStorage.

Q3 MSW scope: GET job, POST application, GET jobs (post-submit invalidation). **MSW cannot test** the wizard's own submit (localStorage) or `CloseJobButton` (Server Action) — neither is a network request.

Q4 naming: rewrote implementation-named tests (e.g. "currentStep equals 'schedule'", "calls setItem with key", "renders 3 divs") into behaviour names ("advances to the schedule step", "a returning user resumes where they left off").

The test that surprised me. "Advance to step 2" flaked only under full-suite load. It revealed that the wizard's `Next` handler validates *asynchronously* and updates the step *after* the `await`, so the re-render lands outside Testing Library's `act()` window. The fix was in the test (a `clickNext` helper that flushes the continuation inside `act`), not the product — a gap in my mental model of async handlers + `act`, not a bug.

6 · Test code — wizard navigation (explained)

src/test/ApplicationWizard.test.tsx

The two helpers everything leans on

```
// Render the wizard with a signed-out-of-profile candidate so step 1 is empty.
function renderWizard(userName = "") {
  return renderWithProviders(
    <JobApplicationWizard job={JOB} user={{ name: userName, email: "alice@example.com" }} />;
  )
}

// Click Next, THEN flush the async validate-and-advance inside act().
async function clickNext(user) {
  await user.click(screen.getByRole("button", { name: /next/i }));
  await act(async () => { await new Promise((r) => setTimeout(r, 0)); });
}

// Fill the minimum required step-1 fields so validation passes.
async function fillStep1(user) {
  await user.type(screen.getByLabelText("Full name(s)", "Alice"));
  await user.type(screen.getByLabelText("Surname", "Candidate"));
  fireEvent.change(screen.getByLabelText("Date of birth"), { target: { value: "1995-06-15" } });
}
```

- **Why `clickNext` flushes inside `act()`** : the wizard's `Next` handler does `await trigger()` then updates step — that re-render lands *after* the click event, outside Testing Library's `act()` window. Flushing a `setTimeout(0)` inside `act` makes the advance deterministic instead of a race (this was the “test that surprised me”).
- **Why `getByLabelText`** : it only finds an input if the `<label>` is correctly associated — so the query doubles as an accessibility assertion.
- **Why `fireEvent` for the date:** `userEvent.type` is unreliable on `type="date"` ; the brief explicitly allows `fireEvent` for inputs `userEvent` can't handle.

The four behaviour tests

```
beforeEach(() => window.localStorage.clear()); // every test is independent

it("renders the step 1 heading on mount (smoke)", () => {
  renderWizard();
  expect(screen.getByRole("heading", { name: "Personal information" })).toBeInTheDocument();
});

it("blocks advancement when required step 1 fields are empty", async () => {
  renderWizard(); const user = userEvent.setup();
  await clickNext(user);
  expect(screen.getByText("Please enter your full name(s).")).toBeInTheDocument(); // inline errors
  expect(screen.getByRole("heading", { name: "Personal information" })).toBeInTheDocument(); // still step 1
});

it("advances to step 2 when step 1 required fields are filled", async () => {
  renderWizard(); const user = userEvent.setup();
  await fillStep1(user); await clickNext(user);
  expect(screen.getByRole("heading", { name: "Contact details" })).toBeInTheDocument();
});

it("Back button preserves step 1 values", async () => {
  renderWizard(); const user = userEvent.setup();
  await fillStep1(user); await clickNext(user);
  await user.click(screen.getByRole("button", { name: /back/i }));
});
```



```
expect(screen.getByDisplayValue("Alice")).toBeInTheDocument(); // values survived  
});
```

Each test asserts something the *user* sees — a heading, an error message, a field value — never an internal variable. That's what makes them survive refactors of the component's internals.

7 · Test code — auth gate & MSW submit (explained)

Auth gate — `src/test/apply-gate.test.tsx`

```
const SESSION_KEY = "careerhub.session";
beforeEach(() => window.localStorage.clear());

it("shows the sign-in prompt and hides the wizard when not authenticated", async () => {
  renderWithProviders(<ApplyPage />, { session: null });
  expect(await screen.findByText("Sign in to apply")).toBeInTheDocument();
  expect(screen.queryByRole("heading", { name: "Personal information" })).not.toBeInTheDocument();
});

it("renders the wizard when authenticated as a candidate", async () => {
  window.localStorage.setItem(SESSION_KEY, JSON.stringify({ id: "u1", name: "Alice", email: "alice@example.com" }));
  renderWithProviders(<ApplyPage />);
  expect(await screen.findByRole("heading", { name: "Personal information" })).toBeInTheDocument();
});
```

- **Why plant `localStorage`, not mock `useSession`** : the page reads identity from `AuthContext` , which restores its session from the `careerhub.session` key on mount. Planting that key tests the *real* auth mechanism end-to-end.
- **`queryByRole` vs `findByRole`** : `query...` returns `null` (used to assert *absence*); `find...` waits for async appearance.

MSW submit — `src/test/ApplicationForm.test.tsx`

```
const API = process.env.NEXT_PUBLIC_API_URL; // same env var the API client reads

it("happy path: shows the confirmation after a successful submission", async () => {
  renderWithProviders(<ApplicationForm jobId="job-1" jobTitle="Senior Frontend Engineer" />);
  await fillAllFields(user);
  await user.click(screen.getByRole("button", { name: /submit application/i }));
  expect(await screen.findByRole("heading", { name: "Application submitted" })).toBeInTheDocument();
});

it("error path: keeps the entered values when the API returns 500", async () => {
  server.use(http.post(`${API}/api/applications`, () => new HttpResponse(null, { status: 500 }))); // per-test
  await fillAllFields(user);
  await user.click(screen.getByRole("button", { name: /submit application/i }));
  expect(await screen.findByRole("alert")).toBeInTheDocument(); // server error shown
  expect(screen.getByDisplayValue("Alice Candidate")).toBeInTheDocument(); // NOT reset
});
```

- **Real HTTP, not a stubbed function**: MSW intercepts the actual `fetch` to `/api/applications` — the component, TanStack Query mutation, and network layer all run for real.
- **`server.use(...)`** overrides a handler for one test; `afterEach` resets it so the 500 can't leak into the next test.

8 · Test code — dialog, drafts & the shared harness

AlertDialog — `src/test/CloseJobButton.test.tsx`

```
vi.mock("@/app/actions/closeJob", () => ({ // MSW can't intercept a Server Action → mock it
  closeJobListing: vi.fn(async () => ({ status: "success", jobTitle: "Senior Frontend Engineer" }))) });

it("opens the AlertDialog when the close button is clicked", async () => {
  renderWithProviders(<CloseJobButton jobId="job-1" currentStatus="Active" />);
  await user.click(screen.getByRole("button", { name: "Close listing" }));
  expect(await screen.findByRole("heading", { name: "Close this listing?" })).toBeInTheDocument();
});

it("calls the close action and shows the closed state when confirmed", async () => {
  await user.click(screen.getByRole("button", { name: "Close listing" }));
  const dialog = await screen.findByRole("alertdialog");
  await user.click(within(dialog).getByRole("button", { name: "Close listing" })); // scope: label shared
  expect(vi.mocked(closeJobListing)).toHaveBeenCalledTimes(1);
  expect(await screen.findByText(/Closed/)).toBeInTheDocument();
});
```

`within(dialog)` scopes the query inside the Radix portal because the trigger and the confirm button share the label "Close listing". `screen` searches the whole document (portals included), so no special handling is needed to find dialog content.

Draft round-trip — `src/test/draft-persistence.test.tsx`

```
it("restores a saved draft + banner on mount", async () => {
  plantDraft({ fullNames: "Restored", surname: "Draftson" }); // write REAL localStorage first
  renderWizard();
  expect(await screen.findByText(/restored your saved draft/i)).toBeInTheDocument();
  expect(screen.getByDisplayValue("Restored")).toBeInTheDocument(); // value came back into the field
});
```

Uses the **real jsdom localStorage** (no spy): the test proves the value actually survives and repopulates the visible field — something a `vi.spyOn` on `setItem` could never show.

The shared harness every file imports — `src/test/utils.tsx` + `setup.ts`

```
// utils.tsx — one render function wraps the providers + controls the session
vi.mock("next-auth/react", () => ({ useSession: vi.fn(), /* ... */ }));
export function renderWithProviders(ui, { session = candidateSession, searchParams = "" } = {}) {
  vi.mocked(useSession).mockReturnValue({ data: session, status: session ? "authenticated" : "unauthenticated" });
  return render(ui, { wrapper: ({ children }) =>
    <NuqsTestingAdapter><QueryClientProvider client={makeClient()}><AuthProvider>{children}</AuthProvider></QueryClientProvider>
  });
}

// setup.ts — global lifecycle for ALL files
beforeAll(() => server.listen()); // MSW on
afterEach(() => { server.resetHandlers(); window.history.replaceState(null, "", "/"); }); // reset handlers +
afterAll(() => server.close());
```

9 · PowerShell runbook — every command to run it

Copy-paste, top to bottom, from the project root in **PowerShell**.

A · Frontend (this repo)

```
# install dependencies (first time only)
npm install

# run the dev server → http://localhost:3000
npm run dev

# production build + start (verifies the whole app compiles)
npm run build ; if ($?) { npm run start }
```

B · Backend + database via Docker (in the careerhub-api repo)

```
# from the careerhub-api repository root (NOT this frontend repo):
docker compose up --build          # API on http://localhost:8080 + PostgreSQL
docker compose ps                  # confirm both containers are healthy
docker compose logs -f api         # follow API logs
docker compose down                # stop everything
```

The backend is a separate repo, not in this workspace. Point the frontend at it with
`NEXT_PUBLIC_API_BASE_URL=http://localhost:8080` in `.env.local`, then restart `npm run dev`.

C · Tests

```
# run once and exit (what CI runs)
npm run test:run

# watch mode while developing
npm test

# a single file
npx vitest run src/test/ApplicationWizard.test.tsx

# type-check (zero errors expected)
npx tsc --noEmit
```

D · Prove it changed (assignment's "Proving It Changed")

```
# 1) remove the validation guard in next() → the "blocks advancement" test fails
# 2) override the POST handler to 500 in the happy-path test → it fails
# 3) comment out afterEach(() => server.resetHandlers()) → the 500 leaks between tests
# restore each → green again
```

E · CI (GitHub Actions)

```
git add . ; git commit -m "test: ci" ; git push    # Actions tab shows a run
```

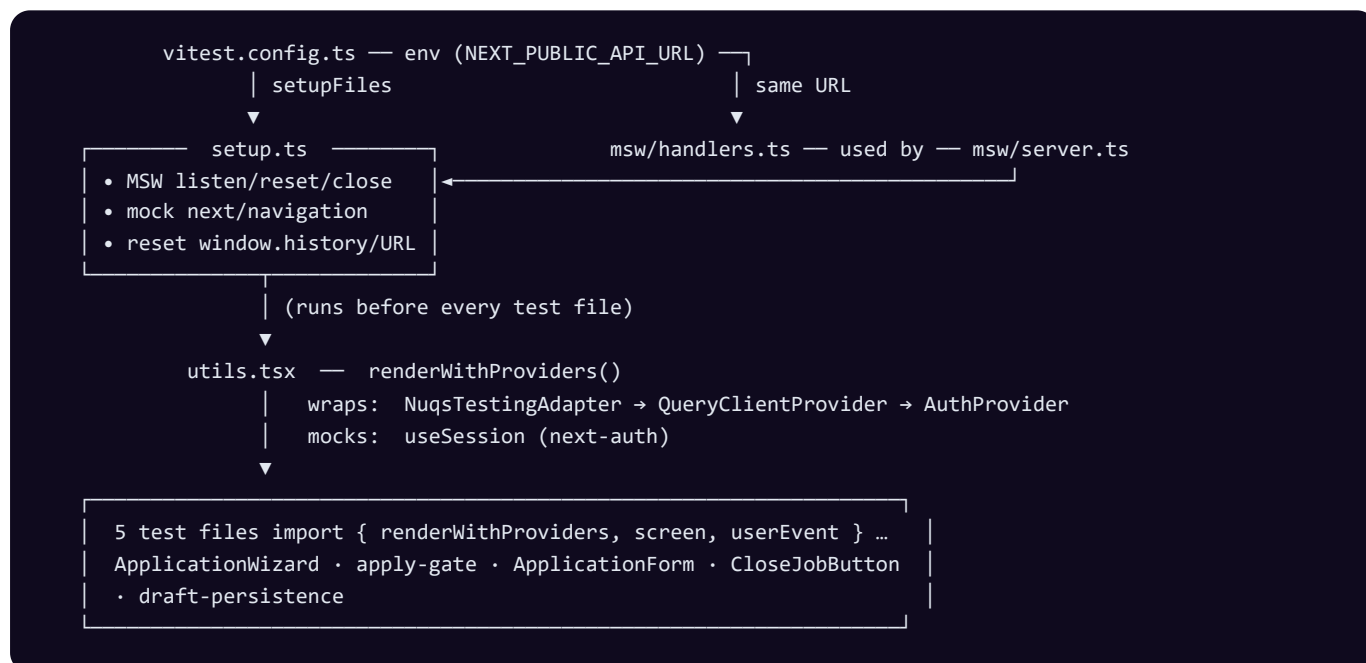
```
# Workflow: .github/workflows/test.yml - push/PR to main, Node 20, npm ci, npm run test:run
```

10 · How everything connects (the whole picture)

Feature code → what the tests pin down

Feature file	Verified by	The contract that's locked
<code>JobApplicationWizard.tsx</code>	<code>ApplicationWizard</code> + <code>draft-persistence</code>	can't advance while invalid; Back keeps answers; drafts round-trip
<code>apply/[jobId]/page.tsx</code>	<code>apply-gate</code>	no candidate ⇒ sign-in prompt, never the form
<code>ApplicationForm.tsx</code>	<code>ApplicationForm</code> (MSW)	success clears the form; a 500 keeps the user's input
<code>CloseJobButton.tsx</code>	<code>CloseJobButton</code>	destructive action always behind a confirm dialog

Test-infrastructure dependency map



- **One render entry point.** Every test calls `renderWithProviders`, so all five files share the exact same providers, the same fake-session control, and the same query client — no per-test boilerplate, no drift.
- **One env, two readers.** `vitest.config.ts` sets `NEXT_PUBLIC_API_URL`; both the *app's* API client and the *MSW handlers* read it, so a handler can never target a URL the code doesn't call.
- **Global hygiene in one place.** `setup.ts` starts/stops MSW, resets handlers, and clears the URL after each test — that's what keeps tests independent (the "Proving It Changed" leak check).

Final summary

This session shipped a polished **5-step application wizard** (Zod + RHF + nuqs), a conditional **document-upload step** with consent gating, **CareerHubX branding**, and a written **backend contract** — then locked that behaviour in with a **12-test suite** (Vitest + Testing Library + MSW + jsdom) and **GitHub Actions CI**. The throughline: tests describe what a *user* experiences (headings, errors, values, dialogs), not how the code is written — so the build can be refactored freely as long as the experience holds. `tsc --noEmit` is clean; `npm run test:run` is 12/12 green.

11 · Five key things to take away

1 · Test behaviour, not implementation

Assert what shows on screen — `getByRole("heading", ...)`, visible error text, `getByDisplayValue` — never an internal state variable or a class name. A good test name reads like a product spec ("Back button preserves step 1 values"), so the suite doubles as living documentation and survives refactors.

2 · Query the way a user (and a screen reader) would

`getByLabelText` only succeeds when a `<label>` is correctly tied to its input, and `getByRole` relies on semantic HTML + ARIA. Writing tests this way *forces* accessible markup — the test fails if labels aren't associated. `queryBy...` asserts absence, `findBy...` waits for async appearance.

3 · Async UI + `act()` is the #1 source of flakes

The wizard's `Next` validates *asynchronously* then updates state, so the re-render lands outside Testing Library's `act()` window and a naïve assertion races it. Flush the continuation (`await act(async () => ...)`) before asserting. Lesson: when a click triggers async work, give the component a chance to settle inside `act` — don't just assert immediately.

4 · Mock at the boundary that matches the seam

Choose the tool to fit how the code actually talks to the outside world: **MSW** for real HTTP (`ApplicationForm` 's `POST`); `vi.mock` for a Server Action (`CloseJobButton`) or a hook (`useSession`); **real jsdom localStorage** for a storage round-trip (drafts). MSW can *only* see network requests — it can't test a localStorage submit or a Server Action.

5 · Isolation is a feature — reset all shared state between tests

Tests must not leak into each other: `server.resetHandlers()` (so a 500 override doesn't bleed forward), `localStorage.clear()` in `beforeEach`, and resetting `window.history` (nuqs writes `?step=` to the URL and jsdom keeps it). When a test only passes because of a previous one, the suite is lying to you.

Project-level takeaway. The frontend is wired to the *real* backend contract but falls back to `localStorage` until the API ships — so the wizard works today and will "just work" against `careerhub-api` once the endpoints in `docs/BACKEND-GAPS.md` exist. The tests gave us the safety net to make that swap without fear of regressing the application flow.